

SOP 50 — Publish

Camada A.N.I.: Instruments-Tools (publish_payload.py) · Versão: 1.0 (Protocol 0, 2026-09-09)
 Golden Rule: mudou a lógica → atualiza o SOP → só então o código. · Regra BLAST: **Complete** =
payload no destino cloud.

Objetivo

Publicar o payload **aprovado** no destino final: registro canônico na cloud (Google Sheets aba `daily_payload` / Supabase) e, após a Phase S, a home HTML estática no ar. **Nunca** publicar a partir de `.tmp/` sem aprovação; **nunca** publicar payload em `status=draft`.

Quando roda

- Cron publish 07:45, se a decisão do editor (SOP 40) foi `approved`.
- Ticks de jogo são exceção explícita: publicam sem aprovação (SOP 31), porque vêm da API, não da IA.

Input

- `DeliveryPayload.Daily` com `status=approved` (decisão do editor em `.tmp/queue/`).
- Credenciais de cloud em `.env` (Sheets/Supabase).
- (Fase S) template da home HTML em `public/` para regenerar a página estática.

Output

- Registro canônico: aba `daily_payload` (Sheets) ou tabela (Supabase) — payload JSON do dia com `status=published` e timestamp.
- Home pública atualizada (fase S): `public/index.html` renderiza o payload approved (estático, mobile first, 4 seções, rodapé “Veículo independente. Não somos o Flamengo.”).
- JSON de relatório:

```
{ "date": "...", "cloud": "ok", "home_html": "ok|pending", "published_at": "..." }
```

Passos determinísticos

1. Ler payload e decisão da fila. **Gate:** só publica se `decision == "approved"` OU (`AUTO_PUBLISH=true` configurado pelo editor E payload válido). Qualquer outro estado → aborta com `status=draft` preservado (não é erro de execução; é regra).
2. Revalidar o payload contra o schema `DeliveryPayload.Daily` (última barreira local).
3. Escrever o registro canônico na cloud (aba `daily_payload`): payload + `status: "published"` + `published_at` (−03:00).
4. **Confirmar a escrita na cloud** (re-ler/confirmar resposta da API). Sem confirmação → retry com backoff (3 tentativas) e **não** reportar Complete.
5. (Fase S) Regenerar `public/index.html` com o payload publicado e publicar no destino estático (Cloudflare Pages / GitHub Pages / bucket).
6. Atualizar `progress.md` com a confirmação e reportar o resumo.

7. Se uma seção do payload estava vazia, ela já existe com `items: [] / status: "empty"` — nunca omitir chave na escrita (invariante de schema).

Edge cases

- Cloud indisponível após retries → payload permanece `approved` na fila; reportar `cloud: "failed"`; **não** marcar Complete; alertar editor via webhook (SOP 90/40). Tentar de novo no próximo ciclo sem republicar conteúdo.
- Payload draft tentando publicar (bug ou reação tardia) → abortar com log; nunca forçar.
- Escrita parcial (linha criada, confirmação perdida) → upsert idempotente por `date`; conferir antes de duplicar.
- `AUTO_PUBLISH=true` não anula a necessidade de payload válido e de seções no schema.
- Fase S ausente (HTML ainda não existe) → publicar só a cloud e reportar `home_html: "pending"` (a home entra na Phase S).

Rate limits / limites conhecidos

- Sheets/Supabase: escrita em lote com backoff em 429/5xx (valores reais fixados na Phase L).
- Publicação estática: 1 deploy por publicação aprovada (não spammar o host).

Testes

- Fixture: payload draft → publish **nega**; payload approved → publish grava na cloud de teste e retorna `cloud: "ok"`.
- Phase L: `tools/ping_sheets.py` valida o caminho da aba `daily_payload`.
- Critério BLAST: só reportar sucesso com confirmação da cloud (ver `progress.md`).